```
0002:
0003: import Link from "next/link";
0004: import { ReactNode } from "react";
0005:
0006: import { StatusType } from "@/lib/types/ip";
0007: import { STATUS_LABELS } from "@/lib/helper/status-labels";
0008: import { formatDate, formatDateTime } from "@/lib/helper/format-date";
0009: import { useGetMultipleApplicationStatuses } from "@/hooks/status/useGetMultipleApplicationStatuses";
0010: import { useGetMultipleAppById } from "@/hooks/applications/useGetMultipleApplicationById";
0011: import { IprStatusType } from "@/lib/types/status";
0012: import { Clock3 } from "lucide-react";
0013:
0014: interface PropsInterface {
0015:   applicationIds: string[];
0016: }
0017:
0018: type StatusListItem = {
0019:   applicationId: string;
0020:   projectTitle: string;
0021:   status: IprStatusType["Row"];
0022:   timestamp: number;
0023: };
0024:
0025: export default function StatusUpdatesPanel(props: PropsInterface) {
0026:   const { applicationIds } = props;
0027:
0028:   const { statuses, isLoading: isStatusesLoading } =
0029:     useGetMultipleApplicationStatuses({
0030:       applicationIds,
0031:       isLatest: true,
0032:     });
0033:
0034:   const { applications } = useGetMultipleAppById({
0035:     applicationIds,
0036:   });
0037:
0038:   if (isStatusesLoading.length > 0 && isStatusesLoading.every(Boolean)) {
0039:     return <StatusUpdateContainer>Fetching statuses...</StatusUpdateContainer>;
0040:   }
0041:
0042:   if (
0043:     !statuses ||
0044:     statuses.length < 1 ||
0045:     !applications ||
0046:     applications.length < 1
0047:   ) {
0048:     return (
0049:       <StatusUpdateContainer>
0050:         <EmptyStateText>No status history available.</EmptyStateText>
0051:       </StatusUpdateContainer>
0052:     );
0053:   }
0054:
0055:   const statusList: StatusListItem[] = statuses
0056:     .map((status, i) => {
0057:       if (Array.isArray(status)) return null;
0058:
0059:       const application = applications[i];
0060:       if (!status || !application || !status.created_at) return null;
0061:
0062:       const timestamp = Date.parse(status.created_at);
0063:       if (isNaN(timestamp)) return null;
0064:
0065:       return {
0066:         applicationId: application.id,
0067:         projectTitle: application.project_title,
0068:         status,
0069:         timestamp,
0070:       };
0071:     })
0072:     .filter((item): item is StatusListItem => item !== null)
0073:     .sort((a, b) => b.timestamp - a.timestamp);
```

```
0074:
0075:   return (
0076:     <StatusUpdateContainer>
0077:       <div className="mt-3 max-h-[24rem] overflow-y-auto pr-1">
0078:         <div className="relative pl-5">
0079:           <div className="absolute top-1 bottom-1 left-[7px] w-px bg-[var(--color-gray-200)]" />
0080:
0081:           <div className="space-y-3">
0082:             {statusList.map((item) => {
0083:               const label =
0084:                 STATUS_LABELS[item.status.status_type as StatusType] ??
0085:                 item.status.status_type;
0086:
0087:               return (
0088:                 <TimelineItem
0089:                   key={`${item.applicationId}-${item.status.created_at}`}
0090:                   applicationId={item.applicationId}
0091:                   projectTitle={item.projectTitle}
0092:                   label={label}
0093:                   createdAt={item.status.created_at!}
0094:                   note={item.status.note}
0095:                   deadline={item.status.deadline}
0096:                 />
0097:               );
0098:             })}
0099:           </div>
0100:         </div>
0101:       </div>
0102:     </StatusUpdateContainer>
0103:   );
0104: }
0105:
0106: interface TimelineItemProps {
0107:   applicationId: string;
0108:   projectTitle: string;
0109:   label: string;
0110:   createdAt: string;
0111:   note?: string | null;
0112:   deadline?: string | null;
0113: }
0114:
0115: function TimelineItem({
0116:   applicationId,
0117:   projectTitle,
0118:   label,
0119:   createdAt,
0120:   note,
0121:   deadline,
0122: }: TimelineItemProps) {
0123:   return (
0124:     <div className="relative">
0125:       <div className="absolute top-2.5 -left-5 h-3.5 w-3.5 rounded-full border border-[var(--color-gray-300)]
      | bg-white">
0126:         <div className="absolute top-1/2 left-1/2 h-1.5 w-1.5 -translate-x-1/2 -translate-y-1/2 rounded-full
      | bg-[var(--color-gray-400)]" />
0127:       </div>
0128:
0129:       <div className="rounded-xl border border-[var(--color-gray-200)] bg-white px-3 py-2.5
      | hover:border-[var(--color-gray-300)]">
0130:         <div className="flex items-start justify-between gap-3">
0131:           <div className="min-w-0 flex-1">
0132:             <div className="flex items-start justify-between gap-3">
0133:               <Link
0134:                 href={`/applications/${applicationId}`}
0135:                 className="truncate text-sm font-medium text-[var(--color-gray-900)] hover:text-[var(--color-
      | brand-600)]"
0136:               >
0137:                 {projectTitle}
0138:               </Link>
0139:
0140:               <p className="shrink-0 text-[11px] text-[var(--color-gray-500)]">
0141:                 {formatDateTime(createdAt)}
```

```
              </p>
            </div>

            <div className="mt-1 flex items-center gap-2">
              <p className="text-xs font-medium text-[var(--color-gray-700)]">
                {label}
              </p>

              <span className="h-1 w-1 rounded-full bg-[var(--color-gray-300)]" />

              <Link
                href={`/techgen/view-application?applicationID=${applicationId}`}
                className="text-xs text-[var(--color-brand-600)] hover:text-[var(--color-brand-700)]"
              >
                View
              </Link>
            </div>

            {note && (
              <p className="mt-2 line-clamp-2 text-xs text-[var(--color-gray-600)]">
                {note}
              </p>
            )}

            {deadline && (
              <p className="mt-1 text-xs text-[var(--color-error-600)]">
                Deadline: {formatDate(deadline)}
              </p>
            )}
          </div>
        </div>
      </div>
    );
}

function StatusUpdateContainer({ children }: { children: ReactNode }) {
  return (
    <div className="rounded-2xl border border-[var(--color-gray-200)] bg-white p-4">
      <div className="mb-4">
        <div className="flex items-center gap-2">
          <div className="bg-brand-50 flex h-8 w-8 items-center justify-center rounded-lg">
            <Clock3 className="text-brand-600 h-4 w-4" />
          </div>
          <h2 className="text-lg font-semibold text-gray-900">
            Recent Updates
          </h2>
        </div>
        <p className="mt-2 text-sm text-gray-500">
          Latest update for each application
        </p>
      </div>
      {children}
    </div>
  );
}

function EmptyStateText({ children }: { children: ReactNode }) {
  return (
    <div className="flex min-h-[120px] items-center justify-center rounded-xl border border-dashed
    | border-[var(--color-gray-200)] bg-[var(--color-gray-25)] px-4 text-sm text-[var(--color-gray-500)]">
      {children}
    </div>
  );
}
```

```
================================================================================================
FILE: src/hooks/applications/useGetUserApplications.ts
================================================================================================
import { getUserApplicationIds } from "@/services/application/get-user-applications";
import { useQuery } from "@tanstack/react-query";

```

```
0004: export function useGetUserApplicationIds() {
0005:     const { data, isLoading } = useQuery({
0006:         queryKey: ['user-applications'],
0007:         queryFn: getUserApplicationIds
0008:     })
0009:
0010:     return {userApplicationIds: data, isLoading}
0011: }
```

```
================================================================================================
FILE: src/services/application/get-user-applications.ts
================================================================================================
0001: import { supabaseClient as supabase } from "@/lib/supabase";
0002:
0003: export const getUserApplicationIds = async function() {
0004:     const { data: { user } } = await supabase.auth.getUser();
0005:     const userId = user?.id;
0006:
0007:     if (!userId) {
0008:         throw new Error("Cannot find user id");
0009:     }
0010:
0011:     const {data, error} = await supabase
0012:     .schema("private")
0013:     .from("inventors")
0014:     .select("application_id")
0015:     .eq("techgen_id", userId)
0016:
0017:     if (error) {
0018:         throw new Error(error.message);
0019:     }
0020:
0021:     return data;
0022: }
```

```
================================================================================================
FILE: src/hooks/views/useGetDashboardAnalyticsTechgen.ts
================================================================================================
0001: import { getDashboardAnalyticsTechgen } from "@/services/views/get-dashboard-analytics-techgen";
0002: import { useQuery } from "@tanstack/react-query";
0003:
0004: export function useGetDashboardAnalyticsTechgen(
0005: ) {
0006:   const { data, isPending } = useQuery({
0007:     queryKey: ["dashboard-analytics-techgen"],
0008:     queryFn: () => getDashboardAnalyticsTechgen(),
0009:   });
0010:
0011:   return { data, isLoading: isPending };
0012: }
```

```
================================================================================================
FILE: src/services/views/get-dashboard-analytics-techgen.ts
================================================================================================
0001: import { supabaseClient as supabase } from "@/lib/supabase";
0002:
0003: export const getDashboardAnalyticsTechgen = async function (
0004: ) {
0005:   const { data: {user}} = await supabase.auth.getUser();
0006:
0007:   if (!user) {
0008:     throw new Error("User not found.");
0009:   }
0010:
0011:   const { data, error } = await supabase
0012:     .from("v_dashboard_analytics_techgen")
0013:     .select()
0014:     .eq("techgen_id", user.id)
0015:     .order("year", { ascending: false });
0016:
0017:   if (error) {
0018:     throw new Error(error.message);
```

```
0019:   }
0020:
0021:   return data;
0022: };

================================================================================================
FILE: src/lib/dashboard/dashboard-summary.ts
================================================================================================
0001: import { DashboardAnalyticsType } from "@/lib/types/views";
0002: import { ipTypeToTitle } from "../helper/get-ip-title";
0003: import { IP_TYPES } from "../types/ip";
0004:
0005: export const STATUS_ORDER = [
0006:   "filed",
0007:   "pending",
0008:   "granted",
0009:   "withdrawn",
0010:   "downgraded",
0011: ] as const;
0012:
0013: export type DashboardStatus = (typeof STATUS_ORDER)[number];
0014:
0015: export const DASHBOARD_STATUS_LABELS: Record<DashboardStatus, string> = {
0016:   filed: "Filed",
0017:   pending: "Pending",
0018:   granted: "Granted",
0019:   withdrawn: "Withdrawn",
0020:   downgraded: "Downgraded",
0021: };
0022:
0023: export type SummaryTableRow = {
0024:   year: number;
0025:   ipType: string;
0026:   filed: number;
0027:   pending: number;
0028:   granted: number;
0029:   withdrawn: number;
0030:   downgraded: number;
0031:   total: number;
0032: };
0033:
0034: export type SummaryTotals = {
0035:   filed: number;
0036:   pending: number;
0037:   granted: number;
0038:   withdrawn: number;
0039:   downgraded: number;
0040:   total: number;
0041: };
0042:
0043: export const formatIpTypeLabel = (ipType: string) => {
0044:   return (
0045:     ipTypeToTitle(ipType) ??
0046:     ipType
0047:       .split("_")
0048:       .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
0049:       .join(" ")
0050:   );
0051: };
0052:
0053: export const buildSummaryTableRows = (
0054:   filteredData: DashboardAnalyticsType[],
0055: ): SummaryTableRow[] => {
0056:   const groupedRows = new Map<string, Omit<SummaryTableRow, "total">>();
0057:
0058:   filteredData.forEach((item) => {
0059:     if (!item.ip_type || item.year === null) return;
0060:     if (!STATUS_ORDER.includes(item.dashboard_status as DashboardStatus)) {
0061:       return;
0062:     }
0063:
0064:     const year = Number(item.year);
```

```
    const key = `${year}:${item.ip_type}`;

    if (!groupedRows.has(key)) {
      groupedRows.set(key, {
        year,
        ipType: item.ip_type,
        filed: 0,
        pending: 0,
        granted: 0,
        withdrawn: 0,
        downgraded: 0,
      });
    }

    const row = groupedRows.get(key)!;
    const status = item.dashboard_status as DashboardStatus;
    row[status] += Number(item.total ?? 0);
  });

  return Array.from(groupedRows.values())
    .map((row) => ({
      ...row,
      total:
        row.filed + row.pending + row.granted + row.withdrawn + row.downgraded,
    }))
    .sort((a, b) => {
      if (a.year !== b.year) return a.year - b.year;

      const aIndex = IP_TYPES.indexOf(a.ipType as (typeof IP_TYPES)[number]);
      const bIndex = IP_TYPES.indexOf(b.ipType as (typeof IP_TYPES)[number]);

      if (aIndex !== -1 && bIndex !== -1) return aIndex - bIndex;
      if (aIndex !== -1) return -1;
      if (bIndex !== -1) return 1;

      return a.ipType.localeCompare(b.ipType);
    });
};

export const buildSummaryTotals = (rows: SummaryTableRow[]): SummaryTotals => {
  return rows.reduce(
    (acc, row) => {
      acc.filed += row.filed;
      acc.pending += row.pending;
      acc.granted += row.granted;
      acc.withdrawn += row.withdrawn;
      acc.downgraded += row.downgraded;
      acc.total += row.total;
      return acc;
    },
    {
      filed: 0,
      pending: 0,
      granted: 0,
      withdrawn: 0,
      downgraded: 0,
      total: 0,
    },
  );
};
```

==
FILE: src/hooks/status/useGetMultipleApplicationStatuses.ts
==

```
import { getApplicationStatuses } from "@/services/status/get-application-statuses";
import { useQueries } from "@tanstack/react-query";

interface PropsInterface {
    applicationIds: string[],
    isLatest?: boolean;
}

```

```
0009: export function useGetMultipleApplicationStatuses(props: PropsInterface) {
0010:     const {applicationIds, isLatest } = props;
0011:     const validApplicationIds = applicationIds.filter(id => id!=null && id!="");
0012:
0013:     const queries = useQueries(
0014:         {
0015:             queries: validApplicationIds.map((applicationId) => (
0016:             {
0017:                 queryKey: ["multiple-status", applicationId, isLatest],
0018:                 queryFn: () => getApplicationStatuses({applicationId, isLatest}),
0019:             }
0020:             ))
0021:         }
0022:     )
0023:
0024:     return { statuses: queries.map((q) => q.data), isLoading: queries.map((q) => q.isLoading)};
0025: }
```

```
================================================================================================
FILE: src/services/status/get-application-statuses.ts
================================================================================================
0001: import { supabaseClient as supabase } from "@/lib/supabase";
0002:
0003: interface PropsInterface {
0004:     applicationId: string,
0005:     isLatest?: boolean,
0006: }
0007:
0008: export const getApplicationStatuses = async function(props: PropsInterface) {
0009:     const { applicationId, isLatest = false } = props;
0010:
0011:     if (!applicationId) {
0012:         throw new Error("Invalid application id.")
0013:     }
0014:
0015:     const { data, error } = await supabase
0016:     .schema("private")
0017:     .from("ipr_statuses")
0018:     .select()
0019:     .eq("application_id", applicationId)
0020:     .order("created_at", {ascending: false});
0021:
0022:     if (error) {
0023:         throw new Error(error.message);
0024:     }
0025:
0026:     if (isLatest) {
0027:         if (!data[0]) {
0028:             throw new Error("No application with that id.")
0029:         }
0030:
0031:         return data[0];
0032:     }
0033:
0034:     return data;
0035: }
```

```
================================================================================================
FILE: src/hooks/applications/useGetMultipleApplicationById.ts
================================================================================================
0001: import { useQueries, useQuery } from '@tanstack/react-query';
0002: import { getApplicationById } from '@/services/application/get-application-by-id';
0003:
0004:
0005: interface UseGetApplicationByIdProp {
0006:   applicationIds: string[];
0007: }
0008:
0009: export function useGetMultipleAppById(props: UseGetApplicationByIdProp) {
0010:     const { applicationIds } = props;
0011:
0012:     const query =  useQueries(
```

```
0013:         {
0014:             queries: applicationIds.map((applicationId) => (
0015:             {
0016:                 queryKey: ['multiple-application', applicationId],
0017:                 queryFn: () => getApplicationById({id: applicationId}),
0018:             }
0019:             ))
0020:         }
0021:     );
0022:
0023:     return {applications: query.map((q) => q.data), isLoading: query.map((q) => q.isLoading)};
0024: }
```

```
================================================================================================
FILE: src/services/application/get-application-by-id.ts
================================================================================================
0001: import { supabaseClient as supabase } from "@/lib/supabase"
0002:
0003: interface GetApplicationByIdProps {
0004:     id: string;
0005: }
0006:
0007: export const getApplicationById = async (props: GetApplicationByIdProps) => {
0008:     const { id } = props;
0009:     const {data, error} = await supabase.schema("private").from("ipr_applications").select().eq("id",
      | id).single();
0010:
0011:     if (error) {
0012:         throw new Error(error.message);
0013:     }
0014:
0015:     return data;
0016: }
```

```
================================================================================================
FILE: src/lib/helper/get-ip-title.ts
================================================================================================
0001: import { IpType } from "@/lib/types/ip";
0002:
0003:
0004: export const  ipTypeToTitle = (ipType: IpType | null | string): string => {
0005:   switch (ipType) {
0006:     case 'utility_model':
0007:       return 'Utility Model';
0008:     case 'industrial_design':
0009:       return 'Industrial Design';
0010:     case 'trademark':
0011:       return 'Trademark';
0012:     case 'copyright':
0013:       return 'Copyright';
0014:     case 'patent':
0015:       return 'Patent'
0016:     default:
0017:       return '';
0018:   }
0019: }
```

```
================================================================================================
FILE: src/lib/helper/status-labels.ts
================================================================================================
0001:
0002: import { StatusType } from "../types/ip";
0003:
0004: export const STATUS_LABELS: Record<StatusType, string> = {
0005:   draft_classification: 'Draft Classification',
0006:   draft_idf: 'Draft IDF',
0007:   submitted_to_ttbdo: 'Submitted to TTBDO',
0008:   under_ttbdo_review: 'Under TTBDO Review',
0009:   prior_art_search: 'Prior Art Search',
0010:   draft_application: 'Draft Application',
0011:   filed_with_ipophil: 'Filed with IPOPHIL',
0012:   wait_notice_publication: 'Waiting for Notice of Publication',
```

```
0013:   wait_substantive_exam_report: 'Waiting for Substantive Exam Report',
0014:   resolve_ser_defects: 'Resolve SER Defects',
0015:   downgraded_to_um: 'Downgraded to UM',
0016:   wait_notice_of_issuance: 'Waiting for Notice of Issuance',
0017:   req_cert_of_registration: 'Request Certificate of Registration',
0018:   wait_cert_of_registration: 'Waiting for Certificate of Registration',
0019:   registered: 'Registered',
0020:   published: 'Published',
0021:   wait_formality_exam_report: 'Waiting for Formality Exam Report',
0022:   resolve_fer_defects: 'Resolve FER Defects',
0023:   request_revival: 'Request Revival',
0024:   '2nd_publication': '2nd Publication',
0025:   prepare_nice_classification: 'Prepare Nice Classification',
0026:   approve_nice_classification: 'Approve Nice Classification',
0027:   wait_registrability_report: 'Waiting for Registrability Report',
0028:   resolve_rr_defects: 'Resolve RR Defects',
0029:   techgen_sign: 'TechGen Sign',
0030:   chancellor_sign: 'Chancellor Sign the Deed of Assignment',
0031:   notarization: 'Notarization',
0032:   wait_notice_of_action: 'Waiting for Notice of Action',
0033:   discussing_downgrade: 'Discussing Downgrade',
0034:   resolve_additional_requirements: 'Resolve Additional Requirements',
0035:   wait_statement_of_acc: 'Waiting for Statement of Account from IPOPHL',
0036:   pay_fee_application: 'Pay Application Fee',
0037:   mailed_to_ipophl: 'Mailing original Signed Copyright Documents to IPOPHL',
0038:   closed: 'Closed',
0039:   req_for_issuance_of_cert_and_2nd_publication: 'Request for Issuance of Certificate and 2nd Publication',
0040:   endorse_copyright: 'Endorse Copyright Application to the Office of the Chancellor',
0041:   print_copyright_forms: 'Print Copyright Forms',
0042:   submit_to_ovcre: 'Submit to OVCRE for Endoresement',
0043:   request_substantive_exam_report: 'Request Substantive Exam Report',
0044:   removal_from_record: 'Removal from Record',
0045:   expired: 'Expired',
0046: };

================================================================================================
FILE: src/lib/helper/format-date.ts
================================================================================================
0001: import { differenceInCalendarDays, isToday, format } from "date-fns";
0002:
0003: export const toSupabaseDate = (date: Date) => format(date, "yyyy-MM-dd");
0004:
0005: export const formatDate = (date: string | Date) =>
0006:   new Date(date).toLocaleDateString(undefined, {
0007:     month: "short",
0008:     day: "numeric",
0009:     year: "numeric",
0010:   });
0011:
0012: export const fromSupabaseDate = (s: string) => {
0013:   const [y, m, d] = s.split("-").map(Number);
0014:   return new Date(y, m - 1, d); // local date, no timezone shift
0015: };
0016:
0017: export const toSupabaseTimestamp = (d: Date) => d.toISOString();
0018: export const fromSupabaseTimestamp = (s: string) => new Date(s);
0019:
0020: export const formatTime = (date: string | Date) =>
0021:   new Date(date).toLocaleTimeString(undefined, {
0022:     hour: "2-digit",
0023:     minute: "2-digit"
0024:   })
0025:
0026: export const formatDateTime = (date: string | Date) =>
0027:   new Date(date).toLocaleString(undefined, {
0028:     month: "short",
0029:     day: "numeric",
0030:     year: "numeric",
0031:     hour: "2-digit",
0032:     minute: "2-digit",
0033:   });
0034:
```

```
0035: export const daysDelayed = (targetISO: string) => {
0036:   const diff = differenceInCalendarDays(new Date(), new Date(targetISO));
0037:   return Math.max(diff, 0);
0038: }
0039:
0040: export const formatSmartDate = (iso: string) => {
0041:   const d = new Date(iso);
0042:   return isToday(d) ? `Today at ${formatTime(iso)}` : formatDateTime(iso);
0043: }
```